

Algorithm Study Guide

Pattern Deep Dive: Frequency Bucket Pattern (LeetCode 347, 451, 692)

1. LeetCode 347 - Top K Frequent Elements Medium

IDEA

nums = 1 1 1 2 2 3

Frequency:

1 -> 3 | 2 -> 2 | 3 -> 1

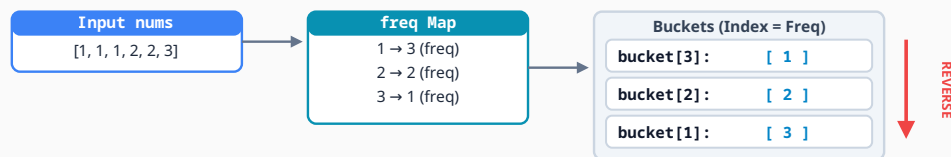
Instead of sorting by frequency, create buckets where:

bucket[1] = numbers appearing once

bucket[2] = numbers appearing twice

bucket[3] = numbers appearing three times

Then iterate from highest frequency to lowest.



IMPLEMENTATION

```
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        unordered_map<int,int> freq;

        // Step 1: Count frequency of every element
        for(int x : nums)
            freq[x]++;

        // Maximum possible frequency = nums.size()
        // bucket[i] stores all numbers having frequency = i
        vector<vector<int>> bucket(nums.size()+1);

        // Step 2: Directly put every number inside bucket corresponding to its frequency
        for(auto &it : freq){
            int number = it.first;
            int frequency = it.second;
            bucket[frequency].push_back(number);
        }

        vector<int> ans;

        // Step 3: Traverse from highest frequency bucket towards lowest
        for(int i = nums.size(); i >= 1 && ans.size() < k; i--){
            for(int x : bucket[i]){
                ans.push_back(x);
                if(ans.size()==k)
                    break;
            }
        }
        return ans;
    }
};
```

Time Complexity

Frequency Count : $O(n)$ Bucket Filling : $O(n)$ Bucket Traversal : $O(n)$ Total = $O(n)$

2. LeetCode 451 - Sort Characters By Frequency Medium

IDEA

Suppose tree

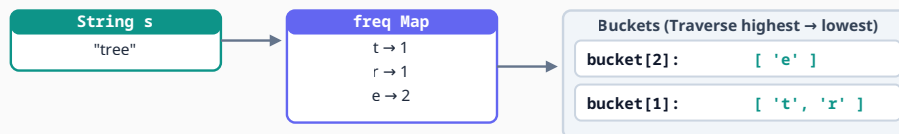
Frequency:

t -> 1 | r -> 1 | e -> 2

Buckets:

bucket[1]: t r | bucket[2]: e

Now iterate backward.



IMPLEMENTATION

```
class Solution {
public:
    string frequencySort(string s) {
        unordered_map<char,int> freq;

        // Count frequency of every character
        for(char c : s)
            freq[c]++;

        // Maximum frequency cannot exceed string length
        vector<vector<char>> bucket(s.size()+1);

        // Put every character inside the bucket representing its frequency
        for(auto &it : freq){
            char ch = it.first;
            int frequency = it.second;
            bucket[frequency].push_back(ch);
        }

        string ans;

        // Traverse buckets from highest frequency towards lowest
        for(int f = s.size(); f >= 1; f--){
            // Every character inside this bucket appeared exactly 'f' times
            for(char ch : bucket[f]){
                // Add character exactly 'f' times
                for(int i=0;i<f;i++)
                    ans.push_back(ch);
            }
        }
        return ans;
    }
};
```

Time Complexity

Overall Time Complexity: $O(n)$

3. LeetCode 692 - Top K Frequent Words Medium

IDEA

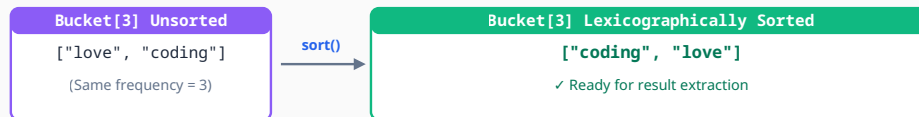
This is slightly different.

If two words have the same frequency, **Need lexicographical order**.

Bucket Sort still works. We simply sort inside each bucket.

Example:

Frequency	Words
3	love, coding
Sort bucket	coding, love



IMPLEMENTATION

```
class Solution {
public:
    vector<string> topKFrequent(vector<string>& words, int k) {
        unordered_map<string,int> freq;

        // Count frequency of every word
        for(string &w : words)
            freq[w]++;

        // bucket[i] stores every word having frequency = i
        vector<vector<string>> bucket(words.size()+1);

        // Place every word into its frequency bucket
        for(auto &it : freq){
            bucket[it.second].push_back(it.first);
        }

        vector<string> ans;

        // Highest frequency first
        for(int f = words.size(); f>=1 && ans.size()<k; f--){
            // If multiple words have same frequency, they must be returned in lexicographical order
            sort(bucket[f].begin(), bucket[f].end());

            for(string &word : bucket[f]){
                ans.push_back(word);
                if(ans.size()==k)
                    break;
            }
        }
        return ans;
    }
};
```

Time Complexity & Heap Note

Frequency count : $O(n)$ Bucket filling : $O(n)$ Sorting inside buckets Worst case : $O(n \log n)$

because all words may have the same frequency. So this bucket solution does not achieve strict $O(n)$ in the worst case. This is why the priority queue (heap) solution is more commonly used in interviews for this problem.

Observe the Common Bucket Pattern

All three problems follow the same template:

```
// Step 1
unordered_map<T,int> freq;
// Count frequency
for(...)
    freq[x]++;

// Step 2
vector<vector<T>> bucket(n+1);
// Frequency becomes bucket index
for(auto &it : freq)
    bucket[it.second].push_back(it.first);

// Step 3: Traverse buckets from highest frequency to lowest
for(int i=n; i>=1; i--) {
    for(auto x : bucket[i]) {
        ...
    }
}
```

The only difference is what you do when you take elements out of the buckets:

Problem	Bucket Stores	While Traversing
347. Top K Frequent Elements	vector<int>	Take first k numbers
451. Sort Characters by Frequency	vector<char>	Append each character frequency times
692. Top K Frequent Words	vector<string>	Sort each bucket lexicographically, then take k words

Key takeaway:
This "frequency bucket" pattern is extremely common in interview questions and is different from the "value range bucket" pattern used in the Maximum Gap problem.